Midterm Study Sheet

EE5203 Microprocessors — what the exam covers (L01–L07), how the questions look, and practice with answers

Basri Erdogan — 2026-2027 Fall

Rule: try every practice item on paper before you open the answers at the end. The exam is written by hand, closed book, 80 minutes. Practice the same way: no compiler, no debugger, a pen and this sheet.

How the exam works

Section	Questions	Points	What you do
A — Trace the execution	4 × 6p	24	Read a short C program. Write exactly what it prints, or the final register / variable values.
B — Multiple choice	5 × 4p	20	One correct answer per question. Concepts from L01–L07.
C — Fill in the blank	2 × 5p	10	A working program with five blanks. One point per blank.
D — Short code	4 questions	46	Write a small function or a loop body. Minimal C; comments welcome.

Every question gives you the facts it needs: the pin, the bit positions, the timer clock, the HAL names. The first page of the exam is a reference box with the HAL and CMSIS names you may use. You do **not** memorise register addresses, bit encodings or function names. You do need to be fluent in the patterns below.

Board facts you may assume: Nucleo-F401RE, LD2 on PA5, B1 on PC13 (external pull-up, pressed = pin low), timer clock 16 MHz.

L01 — Toolchain and evidence

Stage	Tool	Its message when it fails	Typical cause
Compile	compiler	`error: ... undeclared`, `expected ';'`	typo, missing declaration, syntax
Link	linker	`undefined reference to 'f'`	`f` is declared but **defined nowhere** in the build (the `.c` file is not in the build, or the body was never written)
Flash	ST-LINK	connection / target errors	cable, board, wrong project
Run	the board	nothing — you must observe	logic or configuration error

- A clean build proves the **source is valid C**, not that the board does what you want.
- A successful flash proves the firmware **reached the board**, not that it behaves correctly.
- Evidence is an observation: the LED, a variable in the debugger, a register in the SFR view.
- `HAL_Delay(100)` blinks faster than `HAL_Delay(500)` because the loop waits less between toggles.

L02 — Values, loops, decisions, arrays

Rule	Example
Integer division discards the fraction	`17 / 5` is 3; `17 % 5` is 2
`=` stores, `==` compares	`if (x = 1)` is a bug: it stores 1 and is always true
`>` is strict, `>=` is not	`3000 > 3000` is false; `3000 >= 3000` is true
`for (int i = 0; i < N; i++)` runs the body `N` times	`i` is `0` … `N-1` inside; `N` is never used as an index
Valid indices of `uint16_t a[N]`	`0` … `N-1`; `a[N]` is outside the array
Accumulators start at a known value	`sum = 0`, `count = 0`, `maximum = a[0]`
Clamp before you add	`if (a[i] > 4095) a[i] = 4095;` then `sum += a[i];`

How to trace a loop on paper: make a table with one row per repetition and one column per variable. Write the value of every variable **after** the body. Stop when the condition is false. Then compute what is printed.

`printf` in the trace questions. We hardly used `printf` in the labs (the board has no screen); in the exam it is only the way a question asks "what are the values now?". `printf("format", a, b, …)` prints the format text and replaces each `%…` with the next value, in order. Everything else in the format text is printed as it is.

Specifier	Prints	Example
`%d`	a signed integer (`int`)	`printf("%d", -5)` → `-5`
`%u`	an unsigned integer (`uint32_t`, `uint16_t`, `uint8_t`)	`printf("%u", 14995U)` → `14995`
`%02X`	the value in hexadecimal, capital letters, at least 2 digits, zero-padded	`0x4C` → `4C`, `0xC` → `0C`
`%08X`	the same with 8 digits	`0x40C` → `0000040C`
`\n`	ends the line	

- `printf("%d %d\n", 12, 13)` prints `12 13`: one space between the numbers because there is one space in the format text. `"%d,%d"` would print `12,13`.
- An expression is evaluated first: `printf("%d", sum / 6)` prints the result of the integer division, not the expression.
- On the answer line write exactly what is printed: the digits and spaces, no quotes, no units, no variable names.

L03 — Functions, arrays as parameters, strings

- A declaration (in the `.h`) has three parts: **return type, name, parameters**. The definition (in the `.c`) adds the body. The call uses the return value.
- **Parameters are copies.** Changing a parameter never changes the caller's variable. To send a result back, `return` it, and the caller must **store** it: `x = f(x);`.
- An array parameter `uint16_t values[]` is **not a copy** and does not know its own length: the caller passes `count`. Loop with `i < count`.
- Sum many `uint16_t` values into a `uint32_t`: 17×4095 already exceeds 65535.
- `#define SAMPLE_COUNT 8` is text replacement; it is not a variable.
- A C string ends with `'\0'`. `"HIGH"` needs **5** bytes. `{'O','K'}` is not a string. Compare strings with `strcmp(a, b) == 0`, never with `==`.

L04 — Pointers, structures, bits, registers

```
int x = 5;      int *p = &x;      /* p holds the ADDRESS of x           */
*p = 9;                           /* write THROUGH p: x is now 9        */
s.member                          /* member of a structure value        */
p->member   /* same as (*p).member: member through a pointer            */
```

Mask recipes (`n` = bit number, `k` = first bit of a 2-bit field = 2 × pin):

```
r |=  (1U << n);                      /* set bit n, other bits unchanged      */
r &= ~(1U << n);                      /* clear bit n: ~ inverts the mask      */
r ^=  (1U << n);                      /* toggle bit n                         */
if (r & (1U << n)) { /* bit n is 1 */ } /* test bit n: non-zero means set      */
r &= ~(3U << k);  r |= (pattern << k); /* replace a 2-bit field: CLEAR FIRST   */
```

Hex binary: one hex digit = four bits. `0x3 = 0011`, `0xC = 1100`, `0xF = 1111`. `1U << n` is the hex number with a single 1 at bit n: `1U << 4 = 0x10`, `1U << 12 = 0x1000`, `3U << 12 = 0x3000`.

GPIO registers (CMSIS names):

Register	Rule
`RCC->AHB1ENR`	bit 0 = GPIOA clock, bit 1 = GPIOB, bit 2 = GPIOC. **Enable the clock first**; an unclocked port ignores writes.
`GPIOx->MODER`	pin n owns bits 2n+1:2n. `00` input, `01` output, `10` alternate, `11` analog. Read-modify-write.
`GPIOx->BSRR`	bit n (0–15) sets pin n **high**; bit n+16 sets it **low**. Plain `=` is correct: each write is a command.
`GPIOx->IDR`	input level of all 16 pins; `(IDR >> n) & 1U` reads pin n.
`GPIOx->ODR`	current output levels; `^=` toggles a pin.

```
/* worked example: an external LED on PB7 */
RCC->AHB1ENR |= (1U << 1);      /* GPIOB clock (bit 1)                 */
GPIOB->MODER &= ~(3U << 14);    /* clear PB7 field (bits 15:14 = 2·7)  */
GPIOB->MODER |=  (1U << 14);    /* 01 = output                         */
GPIOB->BSRR   =  (1U << 7);     /* PB7 high                            */
GPIOB->BSRR   =  (1U << 23);    /* PB7 low (7 + 16)                    */
GPIOB->ODR   ^=  (1U << 7);     /* toggle PB7                          */
```

Work the same five lines out for LD2 on PA5 yourself: which clock bit, which MODER bits, which BSRR bits? `GPIOB` is a pointer to a `GPIO_TypeDef` structure, so `GPIOB->MODER` is a member reached through a pointer.

L05 — HAL GPIO

```
GPIO_InitTypeDef cfg = {0};
cfg.Pin  = GPIO_PIN_5;              /* a mask: 1U << 5                    */
cfg.Mode = GPIO_MODE_OUTPUT_PP;     /* MODER field 01, OTYPER bit 0       */
cfg.Pull = GPIO_NOPULL;             /* PUPDR field 00                     */
HAL_GPIO_Init(GPIOA, &cfg);         /* &cfg: the function reads your struct */
```

HAL name	Register effect for pin n
`GPIO_MODE_INPUT`	MODER field `00`
`GPIO_MODE_OUTPUT_PP`	MODER field `01`, OTYPER bit n `0`
`GPIO_MODE_IT_FALLING`	MODER `00` + EXTI falling edge + line unmasked
`GPIO_NOPULL` / `GPIO_PULLUP` / `GPIO_PULLDOWN`	PUPDR field `00` / `01` / `10`

- Run-time: `HAL_GPIO_WritePin(port, pin, GPIO_PIN_SET/RESET)`, `HAL_GPIO_TogglePin(port, pin)`, `HAL_GPIO_ReadPin(port, pin)` returns `GPIO_PIN_SET` (high) or `GPIO_PIN_RESET` (low).
- A **floating** input reads noise. A **pull-up** makes the released level high; a **pull-down** makes it low. If the board already has an external resistor, use `GPIO_NOPULL`.
- **Read the wiring, not the word "pressed":** a button that connects the pin to GND gives a *low* level when pressed; a button that connects the pin to 3.3 V gives a *high* level. Work out what `HAL_GPIO_ReadPin` returns from that.
- A button pressed for a long time is one press: detect the **edge** (previous level high, current level low), not the level.

L06 — Interrupts

```
press -> PC13 falls -> EXTI13 edge detect -> EXTI->PR bit 13 = 1 (pending)
      -> EXTI->IMR bit 13 must be 1 -> EXTI15_10_IRQn -> NVIC enable must be 1
      -> CPU saves its place -> vector table entry -> EXTI15_10_IRQHandler()   (processor enters)
      -> HAL_GPIO_EXTI_IRQHandler()  (clears PR)  -> HAL_GPIO_EXTI_Callback(pin)  (you write this)
```

- Two enable switches: `EXTI->IMR` (written by `HAL_GPIO_Init`) and the NVIC (`HAL_NVIC_EnableIRQ`). With the NVIC switch open, EXTI still sets the pending bit, but the handler never runs.

- `EXTI->PR` is **write-1-to-clear**; the HAL helper clears it. Your `button_event` variable is a different flag: `main()` clears it.
- **Callback rules:** check the pin, record the event, return. No `HAL_Delay()`, no loops, no LED code.
- A variable shared with an interrupt is **`volatile`**: otherwise the compiler may keep a stale copy in a register and `main()` never sees the change.
- **Debounce:** accept an edge only if `now - last >= GUARD` (unsigned subtraction, wrap-safe). Update `last` only on an accepted edge. `>=` accepts an edge exactly `GUARD` ms after the last one.

L07 — Timers

$$f_{\text{counter}} = \frac{f_{\text{timer clock}}}{\text{PSC}+1} \qquad T_{\text{update}} = \frac{\text{ARR}+1}{f_{\text{counter}}} \qquad f_{\text{update}} = \frac{f_{\text{timer clock}}}{(\text{PSC}+1)(\text{ARR}+1)}$$

- **Both +1s.** `PSC = 0` divides by 1. `ARR = 99` means 100 counts (0 … 99).
- Many `PSC`/`ARR` pairs give the same period; a smaller `PSC` gives finer resolution.
- `HAL_TIM_Base_Start(&htim)` runs the counter only. `HAL_TIM_Base_Start_IT(&htim)` also enables the update interrupt (`UIE`). Without `_IT` the callback never runs.
- Polling: **test** `TIMx->SR & TIM_SR_UIF`, then **clear** it. Forget the clear and the loop fires continuously.
- Interrupt: the HAL clears `UIF` and calls `HAL_TIM_PeriodElapsedCallback(htim)`. One callback serves all timers: before acting, compare the handle's timer-peripheral member with the timer you expect (the member name is on the exam's reference page).
- Tick pattern: the callback only adds one to the millisecond counter. `main()` schedules with `if ((g_ms - t_task) >= INTERVAL) { t_task = g_ms; /* work */ }`. No `HAL_Delay()` anywhere in that loop.

Practice set

Work on paper. Cover the answers. Part A practises Section A, Part B Section B, Part C Section C and Part D Section D. There are more practice items than exam questions: practise all of them.

Part A — Trace the execution

A1. What is printed?

```c
#include <stdio.h>
#include <stdint.h>

int main(void)
{
    uint16_t readings[5] = {1200, 4300, 3000, 1500, 3000};
    uint32_t sum = 0;
    int high = 0;

    for (int i = 0; i < 5; i++) {
        if (readings[i] > 4095) {
            readings[i] = 4095;
        }
        sum += readings[i];
        if (readings[i] >= 3000) {
            high++;
        }
    }
    printf("%u %d %u\n", sum, high, sum / 5);
    return 0;
}
```

A2. What is printed? Why does the first line not change?

```c
#include <stdio.h>

void double_it(int n)  { n = n * 2; }
int  twice(int n)      { return n * 2; }

int main(void)
{
    int v = 7;
    double_it(v);
    printf("%d\n", v);
    v = twice(v);
    printf("%d\n", v);
    return 0;
}
```

A3. What is printed?

```c
#include <stdio.h>

struct Reading { int id; int value; };

int main(void)
{
    int m = 10;
    int *p = &m;
    *p += 5;
    int n = *p - 3;

    struct Reading r = {1, 50};
    struct Reading *pr = &r;
    pr->value = n;
    r.id = m;
```

```
    printf("%d %d %d %d\n", m, n, r.id, pr->value);
    return 0;
}
```

A4. What is printed (2 hex digits)? If `r` were the **low byte** of `GPIOB->MODER` (pins 3…0; 00 = input, 01 = output, 10 = alternate, 11 = analog), which pin do the last two lines configure and to which mode?

```
#include <stdio.h>
#include <stdint.h>

int main(void)
{
    uint8_t r = 0xF0U;            /* binary 1111 0000 */
    r |=  (1U << 1);
    r &= ~(1U << 4);
    r &= ~(3U << 6);
    r |=  (2U << 6);
    printf("%02X\n", r);
    return 0;
}
```

A5. The guard is 50 ms. Falling edges call the callback at `HAL_GetTick()` = 300, 310, 345, 360, 420, 470 ms. Which edges are accepted, and what is `last` at the end?

```
uint32_t last = 0;
volatile uint8_t event = 0;

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    uint32_t now = HAL_GetTick();
    if (GPIO_Pin == GPIO_PIN_13 && (now - last) >= 50) {
        last = now;
        event = 1;
    }
}
```

A6. `for (int i = 0; i <= 4; i++) printf("%d ", i * i);` — what is printed, and how many times does the body run?

Part B — Concept questions (one answer each)

B1. The build stops at `error: 'mean' undeclared`. Which stage reported it, and what is missing?

B2. The build stops at `undefined reference to 'read_sensor'`. Which stage reported it, and why did compilation succeed?

B3. A pin is configured as input with `GPIO_NOPULL` and nothing is connected to it. What does `HAL_GPIO_ReadPin` return?

B4. An external button connects PA0 to **3.3 V** when pressed, and nothing else is connected. Which pull setting do you choose, and what does a press read?

B5. `HAL_GPIO_Init(GPIOB, &cfg)` ran with `Pin = GPIO_PIN_7`, `Mode = GPIO_MODE_OUTPUT_PP`, `Pull = GPIO_PULLDOWN`. Which bits of MODER and PUPDR changed, and to what?

B6. Which function does the **processor** enter when EXTI line 13 fires, and which function does the **HAL** call for you?

B7. After a press, you pause the program inside the handler. What does `EXTI->PR` bit 13 read, and who will clear it?

B8. The timer clock is 16 MHz, `PSC = 7`, `ARR = 19999`. What is the counter frequency and the update period?

B9. The counter is configured correctly and `HAL_TIM_Base_Start(&htim2)` is called. `TIM2->CNT` changes every time you pause, but the callback never runs. Which bit is 0, and which one call fixes it?

B10. A polling loop tests `UIF` and toggles the LED, and the LED blinks far too fast. Which line is missing?

B11. Why must `g_ms` be declared `volatile`?

B12. `g_ms` is `uint32_t`. Why is `(g_ms - t) >= 500` still correct after `g_ms` wraps from `0xFFFFFFFF` to `0`?

Part C — Fill in the blank

C1. An external LED is on **PB0**. GPIOB's clock enable is `RCC->AHB1ENR` bit 1.

```
void led2_init(void)
{
    RCC->AHB1ENR |= (________);      /* GPIOB clock                  */
    GPIOB->MODER &= ~(________);     /* clear the PB0 mode field     */
    GPIOB->MODER |=  (________);     /* 01 = output                  */
}
void led2_on(void)  { GPIOB->BSRR = (________); }   /* PB0 high */
void led2_off(void) { GPIOB->BSRR = (________); }   /* PB0 low  */
```

C2. Count how many samples are above a limit.

```
int count_above(uint16_t values[], int count, uint16_t limit)
{
    int n = ________;
    for (int i = 0; i < ________; i++) {
        if (values[i] ________ limit) {
            n________;
        }
    }
    return ________;
}
```

C3. B1 on PC13, falling-edge EXTI. Count accepted presses in the callback; `main()` prints every tenth.

```
________ uint32_t press_count = 0;

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == ________) {
        press_count________;
    }
}

/* in main() */
uint32_t shown = 0;
while (1) {
    if (press_count - shown >= ________) {
        shown = press_count;
        printf("%lu presses\n", (unsigned long)________);
    }
}
```

C4. Timer clock 16 MHz. Goal: counter frequency 20 kHz, one update event every 25 ms, by interrupt.

```
htim3.Instance       = TIM3;
htim3.Init.Prescaler = ________;     /* 16 MHz / (PSC + 1) = 20 kHz */
htim3.Init.Period    = ________;     /* 20 kHz / (ARR + 1) = 40 Hz  */
HAL_TIM_Base_Init(&htim3);
________(&htim3);

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == ________) {
        HAL_GPIO_TogglePin(GPIOA, ________);     /* LD2 */
    }
}
```

C5. A button on **PC6** connects the pin to GND when pressed and has no external resistor. GPIOC's clock enable is `RCC->AHB1ENR` bit 2. In MODER, `00` means input; in PUPDR, `01` means pull-up.

```c
void button_init(void)
{
    RCC->AHB1ENR |= (________);      /* GPIOC clock                 */
    GPIOC->MODER &= ~(________);     /* 00 = input: clear the field */
    GPIOC->PUPDR &= ~(________);     /* clear the PC6 pull field    */
    GPIOC->PUPDR |=  (________);     /* 01 = pull-up                */
}
int button_pressed(void)
{
    return (GPIOC->IDR & (________)) == 0U;   /* pressed = pin low */
}
```

C6. **PA9** becomes a UART transmit pin, so its MODER field gets `10` (alternate function). An LED on **PA8** is toggled through `ODR` and switched off through `BSRR`. Pins above 7 work the same way: pin n owns MODER bits $2n{+}1{:}2n$.

```c
GPIOA->MODER &= ~(________);     /* clear the PA9 field              */
GPIOA->MODER |=  (________);     /* write 10: alternate function    */
GPIOA->ODR ____ (________);      /* toggle PA8 (XOR), other pins kept */
GPIOA->BSRR = (________);        /* PA8 low                         */
```

C7. Timer clock 16 MHz. Goal: counter frequency 10 kHz, one update event every 0.5 s, **no interrupt**: `main()` polls the update flag.

```c
htim3.Instance       = TIM3;
htim3.Init.Prescaler = ________;     /* 16 MHz / (PSC + 1) = 10 kHz */
htim3.Init.Period    = ________;     /* 10 kHz / (ARR + 1) = 2 Hz   */
HAL_TIM_Base_Init(&htim3);
________(&htim3);                    /* start the counter only      */

while (1) {
    if (TIM3->SR & ________) {       /* update flag set?            */
        TIM3->SR &= ~TIM_SR_UIF;     /* clear it                    */
        HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0);
    }
}
/* The PB0 LED changes state ____ times per second. */
```

C8. Two buttons with EXTI interrupts: START on PA0 and STOP on PA1. The callback only records which button was pressed; `main()` acts on it.

```c
________ uint8_t start_event = 0;
________ uint8_t stop_event  = 0;

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_0) {
        start_event = 1;
    } else if (GPIO_Pin == ________) {
        stop_event = 1;
    }
}

while (1) {
    if (start_event == 1) {
        start_event = ________;      /* consume the event */
        motor_on();
    }
    if (stop_event == 1) {
        stop_event = 0;
        ________;                    /* switch the motor off */
    }
}
```

Part D — Write the code

D1. Write `uint16_t minimum_sample(uint16_t values[], int count)` returning the smallest element; return `0U` when `count` is 0.

D2. Write `int text_length(char text[])` returning the number of characters before `'\0'`. Then say how many bytes `"STOP"` occupies as a C string.

D3. Write `void gpio_set_pull(GPIO_TypeDef *port, uint32_t pin, uint32_t pull)` that writes the 2-bit `pull` pattern (`00` none, `01` up, `10` down) into the PUPDR field of `pin` without changing other pins. The field of `pin` starts at bit $2 \cdot$ `pin`, so its mask is `3U << (2U * pin)`: clear it first, then OR the shifted pattern in. Then write `uint32_t gpio_read(GPIO_TypeDef *port, uint32_t pin)` that returns 1 when the pin is high and 0 when low, using `IDR`.

D4. With HAL, configure PB0 as a push-pull output (external LED) and PA0 as an input for a button that connects PA0 to 3.3 V when pressed (no external resistor). Fill a `GPIO_InitTypeDef` for each and call `HAL_GPIO_Init`.

D5. PC13 is `GPIO_MODE_IT_FALLING` with its NVIC line enabled. Write the callback (record a PC13 falling edge in a `volatile` flag; no debouncing needed, the callback must not wait or touch the LED) and the `while (1)` body that toggles LD2 on **every third** press. No polling, no `HAL_Delay()`.

D6. `g_ms` is the 1 ms tick counter. The loop below already toggles LD2 every **250 ms** without `HAL_Delay()`:

```
uint32_t t_led = 0;

while (1) {
    if ((g_ms - t_led) >= 250U) {
        t_led = g_ms;
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    }
    /* your code */
}
```

Add a second task with the same pattern so that `log_values()` runs every **1 s**, still with no `HAL_Delay()`. Then explain in one sentence what changes if you write `t_led += 250U` instead of `t_led = g_ms`, and why `g_ms` must be `volatile`.

D7. Without interrupts: write a `while (1)` body that toggles LD2 on every TIM2 update event by **polling** `TIM2->SR` (`TIM_SR_UIF` is bit 0).

D8. Write `int first_over(uint16_t values[], int count, uint16_t limit)` that returns the index of the **first** element greater than `limit`, or `-1` when there is none. `first_over(values, 6, 3000)` with `{1200, 2900, 3100, 3500, 800, 3100}` returns 2.

D9. Write two more port-pointer helpers. `void gpio_toggle(GPIO_TypeDef *port, uint32_t pin)` inverts the pin through `ODR` without changing other pins. `void gpio_set_open_drain(GPIO_TypeDef *port, uint32_t pin, uint32_t on)` writes the **1-bit** `OTYPER` field of `pin` (`1` = open-drain, `0` = push-pull): clear the bit first, then OR the new value in.

D10. Two buttons, two LEDs. B1 (PC13) toggles LD2 (PA5); an external button on PA0 toggles an LED on PB0. Both pins are `GPIO_MODE_IT_FALLING` with their NVIC lines enabled. Write the callback (one `volatile` flag per button) and the `while (1)` body. No polling, no `HAL_Delay()`.

D11. A press of B1 turns LD2 **on**, and LD2 turns **off** by itself 2 s after the **last** press (a new press restarts the 2 s). `g_ms` is the 1 ms tick counter and the callback already sets `volatile uint8_t button_event = 1` on each press. Write the declarations and the `while (1)` body, with no `HAL_Delay()`.

Answers

Part A

A1. `12795 3 2559`. The 4300 is clamped to 4095 before it is added: 1200 + 4095 + 3000 + 1500 + 3000 = 12795. `>= 3000` counts 4095, 3000 and 3000 (three). 12795 / 5 = 2559 exactly.

A2. `7` then `14`. `double_it` changes its own copy `n`; `v` in `main()` is untouched. `twice` returns 14 and the caller stores it with `v = twice(v)`.

A3. `15 12 15 12`. `*p += 5` writes through the pointer, so `m` is 15; `n` is 12; `pr->value = n` sets `r.value`; `r.id = m`.

A4. `A2`. `0xF0 = 1111 0000` → set bit 1 → `1111 0010` (`0xF2`) → clear bit 4 → `1110 0010` (`0xE2`) → clear bits 7:6 → `0010 0010` (`0x22`) → write `10` at bits 7:6 → `1010 0010` (`0xA2`). Bits 7:6 belong to pin 3 (2 × 3 = 6): **PB3, alternate function** (`10`).

A5. Accepted: **300** (300 − 0 50), **360** (360 − 300 = 60), **420** (60), **470** (470 − 420 = 50, and `>=` accepts exactly 50). Rejected: 310 (10) and 345 (45). `last` ends at **470**. Four events.

A6. `0 1 4 9 16` — the body runs **5** times because the condition is `<=`.

Part B

B1. The **compiler**. A declaration of `mean` is missing before its first use (a variable declaration or a function prototype / header include).

B2. The **linker**. The header gave the compiler a declaration, so every call compiled. No object file in the build contains the definition: the `.c` file is not in the build list or the body was never written.

B3. A random, unstable value: the input is floating. Nothing defines the level.

B4. `GPIO_PULLDOWN`, so the released pin reads low. A press connects it to 3.3 V and reads `GPIO_PIN_SET` (active-high).

B5. Pin 7 owns bits 15:14. `MODER[15:14] = 01` (output), `PUPDR[15:14] = 10` (pull-down). Also `OTYPER` bit 7 = 0 (push-pull).

B6. The processor enters `EXTI15_10_IRQHandler()`, the function named in the vector table. The HAL then calls your callback, `HAL_GPIO_EXTI_Callback()`, with `GPIO_PIN_13` as its argument.

B7. It reads **1**: the edge is pending and not yet serviced. The HAL helper `HAL_GPIO_EXTI_IRQHandler()` clears it by writing 1 to the bit, before it calls your callback.

B8. Counter frequency 16 MHz / (7 + 1) = **2 MHz**. Update period (19999 + 1) / 2 MHz = 20000 × 0.5 µs = **10 ms** (100 Hz).

B9. `TIM2->DIER` bit 0 (`UIE`) is 0: the update event never raises an interrupt. Call `HAL_TIM_Base_Start_IT(&htim2)` instead (and keep the NVIC line enabled).

B10. The clear: `TIM2->SR &= ~TIM_SR_UIF;` (or `__HAL_TIM_CLEAR_FLAG(&htim2, TIM_FLAG_UPDATE)`). The flag stays set, so every pass of the loop "sees" an update.

B11. The interrupt changes `g_ms` between two instructions of `main()`. Without `volatile` the compiler may keep a copy in a register and reuse the stale value; `volatile` forces a fresh read from memory every time.

B12. Unsigned subtraction wraps the same way the counter does: for example `3 - 0xFFFFFFFE` gives 5, the true elapsed time. `g_ms >= t + 500` would not survive the wrap.

Part C

C1. `1U << 1`, `3U << 0` (or `3U`), `1U << 0` (or `1U`), `1U << 0`, `1U << 16`.

C2. `0`, `count`, `>`, `++`, `n`.

C3. `volatile`, `GPIO_PIN_13`, `++`, `10`, `press_count` (or `shown`).

C4. `799` (16 MHz / 800 = 20 kHz), `499` (20 kHz / 500 = 40 Hz = 25 ms), `HAL_TIM_Base_Start_IT`, `TIM3`, `GPIO_PIN_5`.

C5. `1U << 2`, `3U << 12`, `3U << 12`, `1U << 12`, `1U << 6`. PC6 owns bits 13:12 in MODER and in PUPDR (2 × 6 = 12); `IDR` has one bit per pin, so the read mask is `1U << 6`.

C6. `3U << 18`, `2U << 18`, `^=`, `1U << 8`, `1U << 24`. PA9 owns bits 19:18 (2 × 9 = 18), and `10` shifted there is `2U << 18`. In BSRR, bit 8 + 16 = 24 drives PA8 low.

C7. `1599` (16 MHz / 1600 = 10 kHz), `4999` (10 kHz / 5000 = 2 Hz = 0.5 s), `HAL_TIM_Base_Start`, `TIM_SR_UIF`, `2` (one toggle per update event).

C8. `volatile`, `volatile`, `GPIO_PIN_1`, `0`, `motor_off()`. Both flags are written by the interrupt and read by `main()`, so both need `volatile`; `main()` clears a flag before acting on it, so a press that arrives during `motor_on()` is not lost.

Part D

D1.

```c
uint16_t minimum_sample(uint16_t values[], int count)
{
    if (count == 0) {
        return 0U;
    }
    uint16_t minimum = values[0];
    for (int i = 1; i < count; i++) {
        if (values[i] < minimum) {
            minimum = values[i];
        }
    }
    return minimum;
}
```

D2.

```c
int text_length(char text[])
{
    int i = 0;
    while (text[i] != '\0') {
        i++;
    }
    return i;
}
```

`"STOP"` occupies **5** bytes: four characters and the `'\0'` terminator.

D3.

```c
void gpio_set_pull(GPIO_TypeDef *port, uint32_t pin, uint32_t pull)
{
    port->PUPDR &= ~(3U << (2U * pin));           /* clear the field */
    port->PUPDR |=  ((pull & 3U) << (2U * pin));  /* write the pattern */
}

uint32_t gpio_read(GPIO_TypeDef *port, uint32_t pin)
{
    return (port->IDR >> pin) & 1U;
}
```

D4.

```c
GPIO_InitTypeDef cfg = {0};

cfg.Pin  = GPIO_PIN_0;              /* PB0: external LED        */
cfg.Mode = GPIO_MODE_OUTPUT_PP;
cfg.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOB, &cfg);

cfg.Pin  = GPIO_PIN_0;              /* PA0: button to 3.3 V     */
cfg.Mode = GPIO_MODE_INPUT;
cfg.Pull = GPIO_PULLDOWN;           /* released reads low       */
HAL_GPIO_Init(GPIOA, &cfg);
```

D5.

```c
volatile uint8_t button_event = 0;
uint8_t  presses = 0;

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_13) {
        button_event = 1;                /* record only */
    }
}

while (1) {
    if (button_event == 1) {
        button_event = 0;                /* consume first */
        presses++;
        if (presses == 3) {
            presses = 0;
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
        }
    }
}
```

The L06 lab's 50 ms guard (`uint32_t now = HAL_GetTick();` and `(now - last_press_ms) >= 50U` in the callback, with `last_press_ms = now;` on an accepted edge) is a good extra, but the exam does not ask for it in the code question; the trace question (A5) covers it.

D6.

```c
uint32_t t_log = 0;                  /* declared next to t_led */

    /* at the "your code" comment, a sibling of the LED block */
    if ((g_ms - t_log) >= 1000U) {
        t_log = g_ms;
        log_values();
    }
```

`t_led += 250U` keeps every deadline on the original 250 ms grid, so a late pass delays one toggle but not the following ones; `t_led = g_ms` re-anchors to the late moment and the lateness accumulates. `g_ms` is written by the TIM2 interrupt outside the flow of `main()`; `volatile` forces every read to go to memory, so the loop sees each new value instead of a copy the compiler kept in a register.

D7.

```c
while (1) {
    if (TIM2->SR & TIM_SR_UIF) {        /* 1. test the flag  */
        TIM2->SR &= ~TIM_SR_UIF;        /* 2. clear it       */
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    }
}
```

The counter must be running (`HAL_TIM_Base_Start(&htim2)` or `TIM2->CR1 |= 1U`).

D8.

```c
int first_over(uint16_t values[], int count, uint16_t limit)
{
    for (int i = 0; i < count; i++) {
        if (values[i] > limit) {
            return i;                /* stop at the first match */
        }
    }
    return -1;                       /* after the loop: none found */
}
```

D9.

```c
void gpio_toggle(GPIO_TypeDef *port, uint32_t pin)
{
```

```
    port->ODR ^= (1U << pin);
}

void gpio_set_open_drain(GPIO_TypeDef *port, uint32_t pin, uint32_t on)
{
    port->OTYPER &= ~(1U << pin);              /* clear the bit   */
    port->OTYPER |=  ((on & 1U) << pin);       /* write 0 or 1    */
}
```

A 1-bit field uses the mask `1U << pin`; a 2-bit field (MODER, PUPDR) uses `3U << (2U * pin)`. The method is the same: clear first, then OR.

D10.

```
volatile uint8_t b1_event = 0;
volatile uint8_t ext_event = 0;

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_13) {
        b1_event = 1;
    } else if (GPIO_Pin == GPIO_PIN_0) {
        ext_event = 1;
    }
}

while (1) {
    if (b1_event == 1) {
        b1_event = 0;
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    }
    if (ext_event == 1) {
        ext_event = 0;
        HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0);
    }
}
```

One callback serves every EXTI line, so it must look at `GPIO_Pin`.

D11.

```
uint32_t t_press = 0;
uint8_t  led_on  = 0;

while (1) {
    if (button_event == 1) {
        button_event = 0;
        t_press = g_ms;                         /* restart the 2 s */
        led_on = 1;
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
    }
    if (led_on == 1 && (g_ms - t_press) >= 2000U) {
        led_on = 0;
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
    }
}
```

The same subtraction test as the scheduler, but the timestamp is set by an event (the press) instead of by the task itself.